Laboratório de Concorrência · Aula 10

Vanessa Beatriz G. Beltrame · RA 24034783-2 · Turma ESOFT6-C
Gerado em 09/10/2026, 20:18:04 · tempo registrado no cronômetro: 39:47

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização de cooperação acontece quando uma tarefa precisa esperar outra realizar algo específico para poder continuar. Já a de competição acontece quando duas ou mais tarefas querem utilizar o mesmo recurso ao mesmo tempo, sendo necessário organizar o acesso para evitar conflitos. Exemplo de cooperação: quando uma pessoa prepara um sanduíche e outra precisa esperar o pão ficar pronto antes de montar o lanche; Exemplo de competição quando duas pessoas querem usar o mesmo computador ao mesmo tempo: é preciso esperar a outra terminar para poder utilizá-lo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1352948

2ª execução (n = 1.000.000)

1031036

3ª execução (n = 1.000.000)

1387979

Saída do programa

```
esperado: 2000000
 obtidos: 1381202
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valore não batem com o valor esperado, nenhum dos três resultados foram iguais ao serem executados. Cada threads ira dar valores diferentes

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Ler: a Thread 1 lê o valor contador = 5. Antes de gravar, a Thread 2 também lê contador = 5.
Somar: a Thread 1 soma 1 e obtém 6. A Thread 2 também soma 1 e obtém 6.
Gravar: a Thread 1 grava 6 no contador. Em seguida, a Thread 2 também grava 6, sobrescrevendo o valor anterior.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

O programa poderia imprimir o valor esperado, 2000, ou um valor menor, dependendo de como as duas threads executaram. Como são menos operações, a condição de corrida pode não aparecer em determinada execução. É perigoso em sistemas reais porque pode passar despercebido nos testes: o programa pode funcionar corretamente várias vezes e apresentar um resultado incorreto em outra execução.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

BBBAAA → 7

ABABBA → 4

ABABAB → 6

AAABBB → 8

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possible: 20
TOTAL = 4: 9 pedidos, por exemplo AABBBA
TOTAL = 6: 9 pedidos, por exemplo AABABB
TOTAL = 7: 1 pedidos, por exemplo BBBAAA
TOTAL = 8: 1 pedidos, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Podem ter 20 ordens possíveis e aparecem AAABBB; BBBAAA ; ABABAB. Apenas um ordem dão o resultado correto que seria 8.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

B lê: a Thread B lê TOTAL = 3.

B calcula: B multiplica 3 por 2 e obtém 6, mas ainda não grava.

A lê: a Thread A lê TOTAL = 3.

A calcula: A soma 1 e obtém 4, mas ainda não grava.

B grava: B grava 6 em TOTAL.

A grava: A grava 4 em TOTAL, sobrescrevendo o valor 6.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a versão com AtomicInteger será a mais rápida, pois utiliza operações atômicas sem precisar bloquear as threads.Nenhuma das três será incorreta, desde que esteja implementada corretamente. As três devem chegar ao resultado esperado de 800.000, evitando a condição de corrida.

Saída do programa

```
esperado: 2000000
sem sincronização: 1950089 em 4 ms
```

```
sincronizado: 2000000 em 46 ms
AtomicInteger: 2000000 em 19 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões que deram o valor correto dfoi a sincronizado e AtomicInteger. A versão correta costuma ser mais lenta porque precisa sincronizar o acesso ao contador, garantindo que as atualizações não sejam perdidas. Isso gera um custo adicional, como o uso de travas no synchronized ou operações atômicas no AtomicInteger, já a versão sem sincronização é mais rápida porque não tem esse controle, mas pode perder atualizações devido à condição de corrida.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Um AtomicInteger em cada saldo garante que cada operação individual seja feita corretamente, mas não garante que a transferência inteira aconteça de forma atômica, seria necessário sincronizar a transferência inteira, usando travas (synchronized ou ReentrantLock) para proteger as duas contas. Além disso, as travas devem ser adquiridas sempre na mesma ordem para evitar um impasse (deadlock), como explicado no slide.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

As mensagens “A terminou” e “B terminou” não vão aparecer, porque ocorre um impasse (deadlock), a thread A bloqueia o recurso X e espera pelo Y, que está com a thread B. Ao mesmo tempo, B bloqueia Y e espera por X, que está com A. Assim, nenhuma consegue continuar nem liberar o recurso que a outra precisa.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A ficou com o recurso X e esperava conseguir o recurso Y, que estava bloqueado pela thread B.
A thread B ficou com o recurso Y e esperava conseguir o recurso X, que estava bloqueado pela thread A.
Assim, as duas ficaram esperando uma pela outra.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina o impasse porque evita que uma thread fique esperando um recurso enquanto outra espera pelo recurso que ela possui. Assim, não ocorre uma espera circular e as threads conseguem continuar. Exemplo real: em um sistema bancário, duas transferências acontecem ao mesmo tempo, uma da conta A para a B e outra da conta B para a A. Se cada transferência bloquear primeiro a conta de origem, pode ocorrer um impasse. Para evitar isso, o sistema bloqueia sempre primeiro a conta de menor número e depois a de maior número, independentemente do sentido da transferência.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores: aproximadamente 200 ms, pois cada trabalhador executa 2 tarefas de 100 ms.
Com 1 trabalhador: aproximadamente 600 ms, pois ele executa as 6 tarefas uma após a outra.
Com 6 trabalhadores: aproximadamente 100 ms, pois cada trabalhador executa uma tarefa ao mesmo tempo.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
trabalhadores 5 fizeram a tarefa 6
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhadores 3 fez a tarefa 4
trabalhadores 4 fez a tarefa 5
trabalhadores 2 fizeram a tarefa 3
6 trabalhadores, 6 tarefas de 100 ms: levaram 100 ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Porque utiliza canais para distribuir as tarefas. Cada tarefa enviada pelo canal é recebida por apenas um trabalhador, evitando que várias goroutines alterem o mesmo dado compartilhado. O WaitGroup garante que o programa espere todos os trabalhadores terminarem antes de mostrar o tempo total.

Saídas (antes e depois da troca)

```
Erro fatal: todas as goroutines estão em estado de repouso - deadlock!

goroutine 1 [chan send]:
principal.principal()
        /tmp/go-compile-1791587035196-8f67gp/main.go:9 +0x36


depois: 1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar mensagens. Quando executa ch <- 1, o envio fica esperando que alguém receba o valor. Porém, o recebimento (<-ch) só aconteceria na linha seguinte, e o programa não consegue chegar até ela. Isso causa um impasse (deadlock). O canal com espaço para uma mensagem (make(chan int, 1)) resolve porque consegue armazenar o valor 1 temporariamente. Assim, o envio termina e o programa consegue executar a próxima linha, receber o valor e imprimi-lo.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa não vai compilar. O compilador Rust detecta que a thread tenta acessar e modificar uma variável pertencente à função main sem garantir que esse acesso seja seguro.

Saídas (o erro e o resultado com move)

```
Caracteres não UTF-8 encontrados no código

0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O programa imprime 0 porque o move faz a thread levar uma cópia da variável contador. A thread incrementa essa cópia para 1, mas a variável original de main continua valendo 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro foi descoberto durante a execução do programa, quando o contador apresentou um valor menor que o esperado devido à condição de corrida.
Em Rust, o compilador detecta o problema antes da execução e impede o compartilhamento inseguro da variável entre threads.
Vantagem do Rust: evita que muitos erros de concorrência cheguem à execução ou à produção.
Custo para quem programa: é necessário seguir regras de propriedade e compartilhamento, usando recursos como Arc<Mutex<T>> ou canais. Isso pode tornar o código mais complexo e exigir mais trabalho no início, mas ajuda a escrever programas mais seguros.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem será A, D, C, B.
Isso acontece porque:
1. A e D: são executadas primeiro, pois fazem parte do código síncrono.
2. C: a promessa (Promise) é executada em seguida, na fila de microtarefas.
3. B: o setTimeout é executado por último, na fila de tarefas.

Saída do programa

```
UM
D
C
B
E: temporizador de 10 ms rodado depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A e D: são impressas primeiro porque o código síncrono é executado diretamente, antes dos temporizadores.
C: é impressa depois do código síncrono porque a promessa (Promise) entra na fila de microtarefas, que é processada antes dos temporizadores.
B: mesmo com setTimeout(..., 0), não é executada imediatamente. Ela precisa esperar o código atual e as microtarefas terminarem.
E: aparece por último, porque a função ocupado(300) bloqueia a única thread por cerca de 300 ms. Durante esse período, o temporizador de 10 ms não consegue executar.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms rodou bem depois porque a função ocupado(300) bloqueou a única thread por aproximadamente 300 ms. Enquanto esse cálculo estava em execução, o laço de eventos não conseguiu atender ao timer, mesmo que os 10 ms já tivessem passado. Em um servidor Node.js, isso significa que um cálculo pesado pode atrasar as respostas para muitos usuários ao mesmo tempo. Enquanto a thread estiver ocupada, novas requisições e outras tarefas ficam esperando, aumentando o tempo de resposta do servidor.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Uma condição de corrida: as duas pessoas tentam comprar o último ingresso ao mesmo tempo, e ambas podem conseguir comprá-lo, causando uma venda duplicada. Para evitar isso, é necessário usar sincronização de competição, garantindo que apenas uma operação por vez verifique a disponibilidade e confirme a compra. Por exemplo, usando uma trava ou uma transação atômica no banco de dados.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Facilidade de começar e terminar as atividades com praticidades.